

Name: Boondirek Kanjanapongporn

GUID: 2833456k

✓ Motivation

Model Architecture:

In the coursework, I decided to train the model with many different architectures. In the first few days, I tried training using `MobileNetV2`. I also tried to test abit of `VGG16_U-Net` architecture. Afterwards, I moved on to using `DeepLabV3_ResNet50` and `DeepLabV3_ResNet100` to train the model instead.

My chosen model architecture for this coursework is trained by `DeepLabV3ResNet100`

Pre-processing:

1. I created `annotation_to_mask()` function which is to convert the `annotation_rgb` images into arrays with pixels as class Id.
2. I use `mask_to_rgb_image()` function to convert the arrays into `annotation_rgb` images to check the correctness
3. Once I know that it is correct, I saved all the arrays and create a folder called `annotation_mask_path` within `damages_data` folder.
4. I need to also add the path into `metadata.csv` so that it can be loaded to `CustomDataModule`
5. Data Augmentation was also added in some my training experiments to see whether it helps the model generalize better or not. Examples of data augmentation I used includes vertical axis flipping and color jitter (adjusting brightness, hues, saturation), along with random cropping that is provided in the coursework.

Training Process:

I applied quite alot of variation for my training processes for each experiment:

1. Loss: I created `MulticlassDiceLoss` and a `CombinedDiceCross-EntropyLoss` to test on some of my experiments to see how it makes a difference compared to `nn.CrossEntropyLoss()`.
2. Hyper-Parameters: In each of the training experiment, I try to adjust hyper-parameter values as a way to explore the hyper-parameter space, like adjusting the batch size, number of epochs, and learning rate.
3. Mixed Precision Training: Most of my experiments undergo mixed precision training (`GradScaler` and `autocast`) so that we can reduce the training time, memory requirements, and computational power needed to train the model without affecting training performance. This way I can also increase the batch size for training (generally, the larger the batch size the better).
4. Learning Rate Scheduler: LRS is also a good way of dynamically adjusting the global learning rate to not overshoot. Often when we train, during the first few epochs we would want to take a larger step to train faster, and later on taking smaller steps to be more detailed. I utilized `ReduceLROnPlateau()` with a patience of 3 and a reduction of a factor of 0.7.
5. Early Stopping: This is very useful due to the fact that I can stop trainig after I know that the model won't have better performance by training anymore. This can help reduce overfitting and reduce time, since I don't have to wait until the end of the training process every time eventhough the performance is not getting better. I utilized the patience of 10.
6. Saving the Best Model: In each training, I have written a code that would save the best performance trained yet, so that after the training process ends, I know that I have the best possible trained model.
7. Cross-Validation: I try to do cross validation by leaving different class out each time to see which group of classes is best as a training set. However, I had only enough time to cross validate 2 classes.

Result Analysis:

The model performance using `DeepLabV3_ResNet101` come out quite satisfying for the limited time available to do the coursework. In can be noticed that the model does a good job at detecting damages, though the predicted class ID could be wrong. As for predicting class ID, it can be seen that damages of type peel (`#FF34FF`) and fold (`#FFA500`) is the easiest to detect among other damages. It can be seen that these two damages are to most frequent identified damages. Following to this is the crack (`#0000A6`) damage, but it is still not that accurate yet.

Feedback and Limitations:

At the end, I am quite satisfy with the work that I have done. Even though it is not the best, I still think that I have done quite alot and tested quite along under a limited time and pressure of this coursework. I think the main limitation is the GPU RAM available in Google Colab. Even though I used mixed precision training, I think I will still get a better result from having a higher batch size and GPU RAM.

Future Work:

For future work, I can try to test on more architectures to compare the results. I can also try other data augmentations like rotation or mixup augmentation. If have more time, I could do more cross-validation experiments as well.

✓ Damages - Deep Learning Coursework 2024

The aim of this coursework will be for you to design, implement and test a deep learning architecture to detect and identify damage in images. Digitization allows to make historical pictures and art much more widely available to the public. Many such pictures have suffered some form of damage due to time, storage conditions and the fragility of the original medium. For example, the image below (A) shows an example of a digitized parchment that has suffered significant damage over time.

The aim of this project is for you to design, implement and evaluate a deep learning model to detect and identify damage present in images.



(A) Image (B) damage labels

(Note that the images will only show once you have downloaded the dataset)

The image labels in this figure (B) identifies a smatter of peeling paint, a large stained area in the bottom left and a missing part on the top left. Each colour in those images corresponds to a different category of damage, including `fold`, `writing` or `burn` marks. You are provided with a dataset of a variety of damaged images, from Parchment to ceramic or wood painting, and detailed annotations of a range of damages.

You are free to use any architecture you prefer, from what we have seen in class. You can decide to use unsupervised pre-training of only supervised end-to-end training - the approach you choose is your choice.

Hand-in date: Friday 15th of March before 4:30pm (on Moodle)

Steps & Hints

- First, look at the data. What are the different type of images (content), what type of material, what type of damage? How different are they? What type of transformations for your data augmentation do you think would be acceptable here?.
- Second, check the provided helper functions for loading the data and separate into training and test set and cross-validation.
- Design a network for the task. What output? What layers? How many? Do you want to use an Autoencoder for unsupervised pre-training?
- Choose a loss function for your network
- Select optimiser and training parameters (batch size, learning rate)
- Optimise your model, and tune hyperparameters (especially learning rate, momentum etc)
- Analyse the results on the test data. How to measure success? Which classes are recognised well, which are not? Is there confusion between some classes? Look at failure cases.
- If time allows, go back to drawing board and try a more complex, or better, model.
- Explain your thought process, justify your choices and discuss the results!

Submission

- submit ONE zip file on Moodle containing:
 - **your notebook**: use `File -> download .ipynb` to download the notebook file locally from colab.
 - **a PDF file** of your notebook's output as you see it: use `File -> print` to generate a PDF.
- your notebook must clearly contains separate cells for:
 - setting up your model and data loader
 - training your model from data
 - loading your pretrained model from github/gitlab/any other online storage you like!
 - testing your model on test data.
- The training cells must be disabled by a flag, such that when running *run all* on your notebook it does
 - load the data
 - load your model
 - apply the model to the test data
 - analyse and display the results and accuracy
- In addition provide markup cell:
 - containing your student number at the top
 - to describe and motivate your design choices: architecture, pre-processing, training regime
 - to analyse, describe and comment on your results
 - to provide some discussion on what you think are the limitations of your solution and what could be future work
- **Note that you must put your trained model online so that your code can download it.**

Assessment criteria

- In order to get a pass mark, you will need to demonstrate that you have designed and trained a deep NN to solve the problem, using sensible approach and reasonable efforts to tune hyper-parameters. You have analysed the results. It is NOT necessary to have any level of accuracy (a network that predicts poorly will always yield a pass mark if it is designed, tuned and analysed sensibly).
- In order to get a good mark, you will show good understanding of the approach and provide a working solution.
- in order to get a high mark, you will demonstrate a working approach of gradual improvement between different versions of your solution.
- bonus marks for attempting something original if well motivated - even if it does not yield increased performance.
- bonus marks for getting high performance, and some more points are to grab for getting the best performance in the class.

Notes

- You are provided code to isolate the test set and cross validation, make sure to keep the separation clean to ensure proper setting of all hyperparameters.
- I recommend to start with small models that can be easier to train to set a baseline performance before attempting more complex one.
- Be mindful of the time!

✓ Housekeeping

```
1 !pip install gdown pytorch_lightning
```

```
Requirement already satisfied: gdown in /usr/local/lib/python3.10/dist-packages (4.7.3)
Collecting pytorch_lightning
  Downloading pytorch_lightning-2.2.1-py3-none-any.whl (801 kB)
      801.6/801.6 kB 7.7 MB/s eta 0:00:00
Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-packages (from gdown) (3.13.1)
Requirement already satisfied: requests[socks] in /usr/local/lib/python3.10/dist-packages (from gdown) (2.31.0)
Requirement already satisfied: six in /usr/local/lib/python3.10/dist-packages (from gdown) (1.16.0)
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (from gdown) (4.66.2)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.10/dist-packages (from gdown) (4.12.3)
Requirement already satisfied: numpy>=1.17.2 in /usr/local/lib/python3.10/dist-packages (from pytorch_lightning) (1.25.2)
Requirement already satisfied: torch>=1.13.0 in /usr/local/lib/python3.10/dist-packages (from pytorch_lightning) (2.2.1+cu121)
Requirement already satisfied: PyYAML>=5.4 in /usr/local/lib/python3.10/dist-packages (from pytorch_lightning) (6.0.1)
Requirement already satisfied: fsspec[http]>=2022.5.0 in /usr/local/lib/python3.10/dist-packages (from pytorch_lightning) (2023.6.0)
Collecting torchmetrics>=0.7.0 (from pytorch_lightning)
  Downloading torchmetrics-1.3.1-py3-none-any.whl (840 kB)
      840.4/840.4 kB 15.0 MB/s eta 0:00:00
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from pytorch_lightning) (24.0)
Requirement already satisfied: typing-extensions>=4.4.0 in /usr/local/lib/python3.10/dist-packages (from pytorch_lightning) (4.10.0)
Collecting lightning-utilities>=0.8.0 (from pytorch_lightning)
  Downloading lightning_utilities-0.10.1-py3-none-any.whl (24 kB)
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.10/dist-packages (from fsspec[http]>=2022.5.0->pytorch_lightning) (3.8.6)
Requirement already satisfied: setuptools in /usr/local/lib/python3.10/dist-packages (from lightning-utilities>=0.8.0->pytorch_lightning) (68.0.0)
Requirement already satisfied: sympy in /usr/local/lib/python3.10/dist-packages (from torch>=1.13.0->pytorch_lightning) (1.12.0)
Requirement already satisfied: networkx in /usr/local/lib/python3.10/dist-packages (from torch>=1.13.0->pytorch_lightning) (3.2.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.10/dist-packages (from torch>=1.13.0->pytorch_lightning) (3.1.3)
Collecting nvidia-cuda-nvrtc-cu12==12.1.105 (from torch>=1.13.0->pytorch_lightning)
  Downloading nvidia_cuda_nvrtc_cu12-12.1.105-py3-none-manylinux1_x86_64.whl (23.7 MB)
      23.7/23.7 MB 43.6 MB/s eta 0:00:00
Collecting nvidia-cuda-runtime-cu12==12.1.105 (from torch>=1.13.0->pytorch_lightning)
  Downloading nvidia_cuda_runtime_cu12-12.1.105-py3-none-manylinux1_x86_64.whl (823 kB)
      823.6/823.6 kB 44.4 MB/s eta 0:00:00
Collecting nvidia-cuda-cupti-cu12==12.1.105 (from torch>=1.13.0->pytorch_lightning)
  Downloading nvidia_cuda_cupti_cu12-12.1.105-py3-none-manylinux1_x86_64.whl (14.1 MB)
      14.1/14.1 MB 65.0 MB/s eta 0:00:00
Collecting nvidia-cudnn-cu12==8.9.2.26 (from torch>=1.13.0->pytorch_lightning)
  Downloading nvidia_cudnn_cu12-8.9.2.26-py3-none-manylinux1_x86_64.whl (731.7 MB)
      731.7/731.7 MB 2.4 MB/s eta 0:00:00
Collecting nvidia-cublas-cu12==12.1.3.1 (from torch>=1.13.0->pytorch_lightning)
  Downloading nvidia_cublas_cu12-12.1.3.1-py3-none-manylinux1_x86_64.whl (410.6 MB)
      410.6/410.6 MB 3.1 MB/s eta 0:00:00
Collecting nvidia-cufft-cu12==11.0.2.54 (from torch>=1.13.0->pytorch_lightning)
  Downloading nvidia_cufft_cu12-11.0.2.54-py3-none-manylinux1_x86_64.whl (121.6 MB)
      121.6/121.6 MB 13.3 MB/s eta 0:00:00
Collecting nvidia-curand-cu12==10.3.2.106 (from torch>=1.13.0->pytorch_lightning)
  Downloading nvidia_curand_cu12-10.3.2.106-py3-none-manylinux1_x86_64.whl (56.5 MB)
      56.5/56.5 MB 29.2 MB/s eta 0:00:00
Collecting nvidia-cusolver-cu12==11.4.5.107 (from torch>=1.13.0->pytorch_lightning)
  Downloading nvidia_cusolver_cu12-11.4.5.107-py3-none-manylinux1_x86_64.whl (124.2 MB)
      124.2/124.2 MB 7.6 MB/s eta 0:00:00
Collecting nvidia-cuspars-cu12==12.1.0.106 (from torch>=1.13.0->pytorch_lightning)
  Downloading nvidia_cuspars-cu12-12.1.0.106-py3-none-manylinux1_x86_64.whl (196.0 MB)
      196.0/196.0 MB 4.0 MB/s eta 0:00:00
Collecting nvidia-nccl-cu12==2.19.3 (from torch>=1.13.0->pytorch_lightning)
  Downloading nvidia_nccl_cu12-2.19.3-py3-none-manylinux1_x86_64.whl (166.0 MB)
      166.0/166.0 MB 10.0 MB/s eta 0:00:00
Collecting nvidia-nvtx-cu12==12.1.105 (from torch>=1.13.0->pytorch_lightning)
  Downloading nvidia_nvtx_cu12-12.1.105-py3-none-manylinux1_x86_64.whl (99 kB)
```

```

1
2
3 import torch
4 import torch.nn as nn
5 import torch.optim as optim
6 import torch.nn.functional as F
7 from torch.utils.data import Dataset, DataLoader
8 from torchvision import transforms
9 import torchvision.models as models
10 from torch.cuda.amp import GradScaler, autocast
11 from torch.optim.lr_scheduler import StepLR
12 from torch.optim.lr_scheduler import ReduceLROnPlateau
13
14 import pytorch_lightning as pl
15 from pytorch_lightning.loggers import WandbLogger
16 from pytorch_lightning.callbacks import LearningRateMonitor, ModelCheckpoint, EarlyStopping
17 from pytorch_lightning import Trainer
18
19 import os
20 import pandas as pd
21 import PIL
22 PIL.Image.MAX_IMAGE_PIXELS = 243748701
23 from PIL import Image
24 import numpy as np
25 import matplotlib.pyplot as plt
26 import gdown
27 import shutil
28 import random
29 import sys
30
31 DEVICE = torch.device("cuda:0") if torch.cuda.is_available() else torch.device("cpu")
32 run_this_cell = False

```

✓ Load dataset

We then load the metadata in a dataframe for convenience

```

1 !pwd
2 !pip install --upgrade --no-cache-dir gdown # Update pip version (default has problems)

/content
Requirement already satisfied: gdown in /usr/local/lib/python3.10/dist-packages (4.7.3)
Collecting gdown
  Downloading gdown-5.1.0-py3-none-any.whl (17 kB)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.10/dist-packages (from gdown) (4.12.3)
Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-packages (from gdown) (3.13.1)
Requirement already satisfied: requests[socks] in /usr/local/lib/python3.10/dist-packages (from gdown) (2.31.0)
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (from gdown) (4.66.2)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.10/dist-packages (from beautifulsoup4->gdown) (2.5)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests[socks]->gdown) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests[socks]->gdown) (3.6)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests[socks]->gdown) (2.0.7)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests[socks]->gdown) (2024.2.2)
Requirement already satisfied: PySocks!=1.5.7,>=1.5.6 in /usr/local/lib/python3.10/dist-packages (from requests[socks]->gdown) (1.7.1)
Installing collected packages: gdown
  Attempting uninstall: gdown
    Found existing installation: gdown 4.7.3
    Uninstalling gdown-4.7.3:
      Successfully uninstalled gdown-4.7.3
  Successfully installed gdown-5.1.0

```

Load zip

```

1 # Original damage data
2 # !gdown 1kdaB53poIJoqdGAXogVcq35EeYazboe5 -O damages.zip
3
4 # Damage data version 2 (with annotation_mask_path v2)
5 # !gdown 1NkIL7eP1yJYlLe-7TWjy7ZPvf065A8Rv -O damages.zip
6
7 # Damage data version 3 (with annotation_mask_path v3)
8 # !gdown 1JUo4znLkJdDX4l0RE-4ni2dKz6KhnLP2 -O damages.zip

```

```

9 # !gdown 1jIBbeVBb58wbPjKF-DvwfiOSTBxnn16v -O damages.zip
10
11 # Damage data version 4 (with annotation_mask_path and trained model)
12 !gdown 1bMwaMKt115v8c3ildX7xtN3W3dNa0bFP -O damages.zip

Downloading...
From (original): https://drive.google.com/uc?id=1jIBbeVBb58wbPjKF-DvwfiOSTBxnn16v
From (redirected): https://drive.google.com/uc?id=1jIBbeVBb58wbPjKF-DvwfiOSTBxnn16v&confirm=t&uuid=bfcfccef-893f-40dd-b65c-8fed939e49d9
To: /content/damages.zip
100% 3.35G/3.35G [01:11<00:00, 47.1MB/s]

```

Unzip damages.zip file

```

1 import zipfile
2
3 # Path to the zip file
4 zip_path = 'damages.zip'
5 extraction_path = '/content/' # This specifies the root directory
6
7 # Extract the zip file
8 with zipfile.ZipFile(zip_path, 'r') as zip_ref:
9     zip_ref.extractall(extraction_path)
10
11 print(f"Extracted all contents to {extraction_path}")

Extracted all contents to /content/

1 # set that to wherever you want to store the data (eg, your Google Drive), choose a persistent location!
2 root_dir = '.'
3 data_dir = os.path.join(root_dir, "damage_data")
4 csv_path = os.path.join(data_dir, 'metadata.csv')
5
6 try:
7     df = pd.read_csv(csv_path)
8
9 except: # if the dataset has not been downloaded yet, do it.
10     zip_path = os.path.join(root_dir, 'damages.zip')
11     gdown.download(id='1JUo4znLkJdDX4l0RE-4ni2dKz6KhnlP2', output=zip_path)
12     shutil.unpack_archive(zip_path, root_dir)
13     df = pd.read_csv(csv_path)

```

This dataframe has the paths of where the dataset images and annotation labels are stored, plus classification labels.

```
1 df
```

	id	material	content	i
0	cljmrkz5n341f07clcuju105j	Parchment	Artistic depiction	./damage_data/image_path/cljmrkz5n341
1	cljmrkz5n341n07clf1u410ed	Parchment	Artistic depiction	./damage_data/image_path/cljmrkz5n341i
2	cljmrkz5n341r07chl93dpre	Parchment	Artistic depiction	./damage_data/image_path/cljmrkz5n341
3	cljmrkz5n341v07cl2gfhd6zj	Parchment	Artistic depiction	./damage_data/image_path/cljmrkz5n341'
4	cljmrkz5n341z07cldbn01un3	Parchment	Artistic depiction	./damage_data/image_path/cljmrkz5n341z
...	...	...	...	...
390	clnofow7i00n2076ubpfodf4d	Wood	Artistic depiction	./damage_data/image_path/clnofow7i00n2
391	clnrm7fvu092q07840tq9zs03	Wood	Artistic depiction	./damage_data/image_path/clnrm7fvu092q
392	clnrm7fvu092r0784a6p2m5li	Wood	Artistic depiction	./damage_data/image_path/clnrm7fvu092rl
393	clnrm7fvu092s0784tgwccewe	Wood	Artistic depiction	./damage_data/image_path/clnrm7fvu092s
394	clnrm7fvu092t0784ltljhiu0	Wood	Artistic depiction	./damage_data/image_path/clnrm7fvu09

395 rows × 7 columns

The images in the dataset are categorised in terms of the type of `material`, meaning what was the original picture on, eg, Parchment, Glass or Textile.

```
1 df['material'].unique()

array(['Parchment', 'Film emulsion', 'Glass', 'Paper', 'Tesserae',
       'Canvas', 'Textile', 'Ceramic', 'Wood'], dtype=object)
```

Moreover, images are also categorised in terms on the `content` of the image, meaning what is depicted: eg, Line art, geometric patterns, etc.

```
1 df['content'].unique()

array(['Artistic depiction', 'Line art', 'Photographic depiction',
       'Geometric patterns'], dtype=object)
```

✓ Labels

Segmentation labels are saved as a PNG image, where each number from 1 to 15 corresponds to a damage class like Peel, Scratch etc; the Background class is set to 255, and the Clean class (no damage) is set to 0. We also provide code to convert these annotation values to RGB colours for nicer visualisation, but for training you should use the original annotations.

```
1 name_color_mapping = {
2     "Material loss": "#1CE6FF",
3     "Peel": "#FF34FF",
4     "Dust": "#FF4A46",
5     "Scratch": "#008941",
6     "Hair": "#006FA6",
7     "Dirt": "#A30059",
8     "Fold": "#FFA500",
9     "Writing": "#7A4900",
10    "Cracks": "#0000A6",
11    "Staining": "#63FFAC",
12    "Stamp": "#004D43",
13    "Sticker": "#8FB0FF",
14    "Puncture": "#997D87",
15    "Background": "#5A0007",
16    "Burn marks": "#809693",
17    "Lightleak": "#f6ff1b",
```

```

18 }
19
20 class_names = [ 'Material loss', 'Peel', 'Dust', 'Scratch',
21                 'Hair', 'Dirt', 'Fold', 'Writing', 'Cracks', 'Staining', 'Stamp',
22                 'Sticker', 'Puncture', 'Burn marks', 'Lightleak', 'Background']
23
24 class_to_id = {class_name: idx+1 for idx, class_name in enumerate(class_names)}
25 # class_to_id['Background'] = 255 # Set the Background ID to 255
26
27 def hex_to_rgb(hex_color: str) -> tuple:
28     hex_color = hex_color.lstrip('#')
29     return tuple(int(hex_color[i:i+2], 16) for i in (0, 2, 4))
30
31 id_to_rgb = {class_to_id[class_name]: hex_to_rgb(color) for class_name, color in name_color_mapping.items()}
32 id_to_rgb[0] = (0,0,0)
33
34 # Create id2label mapping: ID to class name
35 id2label = {idx: class_name for class_name, idx in class_to_id.items()}
36
37 # Create label2id mapping: class name to ID, which is the same as class_to_id
38 label2id = class_to_id
39
40 # Non-damaged pixels
41 id2label[0] = 'Clean'
42 label2id['Clean'] = 0

1 from IPython.display import Markdown
2
3 legend='#### Colour labels for each damage type\n'
4 for damage in class_names:
5     # legend += '- <span style="color: {color}">{damage}</span>.\n'.format(color=name_color_mapping[damage], damage=damage)
6     legend += '- <span style="color: {color}">{color} -> {damage}</span>.\n'.format(color=name_color_mapping[damage], damage=damage)
7 display(Markdown(legend))

```

Colour labels for each damage type

- #1CE6FF -> Material loss.
- #FF34FF -> Peel.
- #FF4A46 -> Dust.
- #008941 -> Scratch.
- #006FA6 -> Hair.
- #A30059 -> Dirt.
- #FFA500 -> Fold.
- #7A4900 -> Writing.
- #0000A6 -> Cracks.
- #63FFAC -> Staining.
- #004D43 -> Stamp.
- #8FB0FF -> Sticker.
- #997D87 -> Puncture.
- #809693 -> Burn marks.
- #f6ff1b -> Lightleak.
- #5A0007 -> Background.

✓ Preprocess Labels

We create functions to mask the annotation rgb images from the annotation_rgb_paths to pixels of class ID label.

```

1 from os import listdir
2 from os.path import isfile, join
3
4 # Print all file names from the given directory
5 def getfilenamesfromdir(path, folder):
6     fullpath = f"{path}/{folder}"
7     return [f for f in listdir(fullpath) if isfile(join(fullpath, f))]

1 # Create a reverse mapping from RGB to class ID
2 rgb_to_id = {hex_to_rgb(color): class_to_id[class_name] for class_name, color in name_color_mapping.items()}
3 rgb_to_id[(0,0,0)] = 0
4
5 # Function to convert an annotation rgb image to a class ID mask
6 def annotation_to_mask(annotation_path):
7     # Open the image and convert to numpy array
8     annotation_image = Image.open(annotation_path)
9     annotation_array = np.array(annotation_image)

```

```

9     annotation_array = np.array(annotation_image)
10
11     # Create an empty array for the mask
12     mask = np.zeros(annotation_array.shape[:2], dtype=np.int32)
13
14     # Populate the mask array with class IDs
15     for rgb, class_id in rgb_to_id.items():
16         mask[np.all(annotation_array == rgb, axis=-1)] = class_id
17
18     return mask
19
20 # Function to mask
21 def mask_annotations():
22     filenames = os.listdir(join(data_dir, 'annotation_rgb_path'))
23     # filenames = getfilenamesfromdir(data_dir, 'annotation_rgb_path')
24
25     # Create a new directory if it does not exist
26     mask_path = join(data_dir, 'annotation_mask_path')
27     isExist = os.path.exists(mask_path)
28     if not isExist:
29         os.mkdir(mask_path)
30
31     # annotation_rgb_path = join(data_dir, 'annotation_rgb_path')
32     for filename in filenames:
33         mask = annotation_to_mask(join(data_dir, 'annotation_rgb_path', filename))
34         np.save(f"{join(mask_path, filename[:-4])}", mask)
35
36 # The new damages.zip already has annotation_mask_path
37 # Uncomment below if you are using original damages.zip
38 # mask_annotations()

```

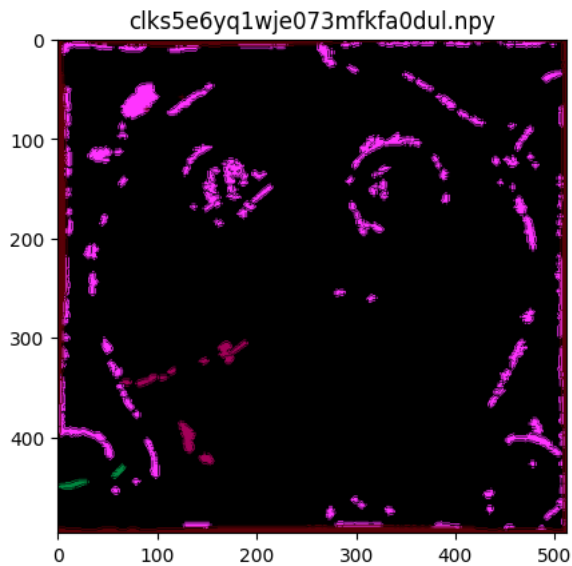
✓ Test Annotation Masking

Check whether we have masked the annotation rgb images correctly

```

1 # Reverse mask image to rgb image for visualisation
2 def mask_to_rgb_image(mask):
3     rgb_image = np.zeros((mask.shape[0], mask.shape[1], 3), dtype=np.uint8)
4
5     for class_id, rgb in id_to_rgb.items():
6         rgb_image[mask == class_id] = rgb
7
8     return rgb_image
9
10 # Load a mask image
11 filename = 'clks5e6yq1wje073mfkfa0dul.npy'
12 annotation_mask_path = join(data_dir, 'annotation_mask_path')
13 annotation_mask = np.load(join(annotation_mask_path, filename))
14
15 rgb_image = mask_to_rgb_image(annotation_mask)
16 # rgb_image = transforms.functional.to_pil_image(annotation_mask)
17
18 # Display the image
19 plt.title(filename)
20 plt.imshow(rgb_image)
21 plt.show()

```



▼ Create dataset splits

Here is an example of how to split the dataset for Leave-one-out cross validation (LOOCV) based on material.

```
1 def create_leave_one_out_splits(df, criterion='material'):
2
3     grouped = df.groupby(criterion)
4     content_splits = {name: group for name, group in grouped}
5     unique_val = df[criterion].unique()
6
7     # Initialize a dictionary to hold the train and validation sets for each LOOCV iteration
8     loocv_splits = {}
9
10    for value in unique_val:
11        # Create the validation set
12        val_set = content_splits[value]
13
14        # Create the training set
15        train_set = pd.concat([content_splits[c] for c in unique_val if c != value])
16
17        # Add these to the loocv_splits dictionary
18        loocv_splits[value] = {'train_set': train_set, 'val_set': val_set}
19
20    return loocv_splits
21
```

For this coursework, we will want to assess the generalisation of the method, so for that we will keep one type of material (Canvas) as test set, and only train on the remaining ones.

```
1 # split the dataset according to material type
2 full_splits = create_leave_one_out_splits(df, 'material')
3
4 # use Canvas as test set
5 test_set = full_splits['Canvas']['val_set']
6
7 # use the rest as training set
8 train_set = full_splits['Canvas']['train_set']
9
10 # prepare a leave-one-out cross validation for the training set
11 loocv_splits = create_leave_one_out_splits(train_set, 'material')
12
13 # identify the different type of image content
14 unique_material = train_set['material'].unique()
15 print(unique_material)
16
17 ['Parchment' 'Film emulsion' 'Glass' 'Paper' 'Tesserae' 'Textile'
18  'Ceramic' 'Wood']
```

To help you, here are some helper functions to help crop and process images.

```
1 def random_square_crop_params(image, target_size):
2     width, height = image.size
3     min_edge = min(width, height)
4
5     # Conditionally set the range for random crop size
6     lower_bound = min(min_edge, target_size)
7     upper_bound = max(min_edge, target_size)
8
9     # Generate crop_size
10    crop_size = random.randint(lower_bound, upper_bound)
11
12    # Check and adjust if crop_size is larger than any dimension of the image
13    if crop_size > width or crop_size > height:
14        crop_size = min(width, height)
15
16    # Generate random coordinates for the top-left corner of the crop
17    x = random.randint(0, width - crop_size)
18    y = random.randint(0, height - crop_size)
19
20    return (x, y, x + crop_size, y + crop_size)
21
22 def apply_crop_and_resize(image, coords, target_size):
23     image_crop = image.crop(coords)
24     image_crop = image_crop.resize((target_size, target_size), Image.NEAREST)
25     return image_crop
```

I create functions to center crop images

```
1 def center_square_crop_params(image, target_size):
2     width, height = image.size
3     min_edge = min(width, height)
4
5     if min_edge >= target_size:
6         x1 = width // 2 - target_size // 2
7         y1 = height // 2 - target_size // 2
8         x2 = x1 + target_size
9         y2 = y1 + target_size
10    else:
11        aspect_ratio = width / height
12        if width < height:
13            new_width = target_size
14            new_height = int(new_width / aspect_ratio)
15        else:
16            new_height = target_size
17            new_width = int(new_height * aspect_ratio)
18
19    # Calculate center crop coordinates after resizing
20    x1 = new_width // 2 - target_size // 2
21    y1 = new_height // 2 - target_size // 2
22    x2 = x1 + target_size
23    y2 = y1 + target_size
24
25    return x1, y1, x2, y2
26
27 def apply_center_crop(image, target_size):
28     crop_coords = center_square_crop_params(image, target_size)
29     # For smaller images, we resize first to ensure the crop size can be achieved
30     if min(image.size) < target_size:
31         # Calculate new size preserving the aspect ratio
32         aspect_ratio = image.width / image.height
33         if image.width < image.height:
34             new_width = target_size
35             new_height = round(new_width / aspect_ratio)
36         else:
37             new_height = target_size
38             new_width = round(new_height * aspect_ratio)
39         image = image.resize((new_width, new_height), Image.Resampling.LANCZOS)
40     return image.crop(crop_coords)
41
42 def apply_center_crop_to_mask(mask_image, target_size):
43     # Apply center cropping to the mask using nearest-neighbor interpolation
44     # Get current size
```

```

45     width, height = mask_image.size
46
47     # Determine if resizing is needed
48     if min(width, height) < target_size:
49         aspect_ratio = width / height
50         if width < height:
51             new_width = target_size
52             new_height = round(new_width / aspect_ratio)
53         else:
54             new_height = target_size
55             new_width = round(new_height * aspect_ratio)
56         resized_mask = mask_image.resize((new_width, new_height), Image.NEAREST)
57     else:
58         resized_mask = mask_image
59
60     # Calculate center crop coordinates and crop
61     crop_coords = center_square_crop_params(resized_mask, target_size)
62     return resized_mask.crop(crop_coords)

```

We also provide a simple class for holding the dataset

Modify:

I also add annotation mask into the CustomDataset

1. CustomDataset Original

```

1 import torch
2 from torch.utils.data import DataLoader, Dataset
3 from torchvision import transforms
4 import random
5 import numpy as np
6 from PIL import Image
7
8 from torchvision import transforms
9
10 class CustomDataset(Dataset):
11     def __init__(self, dataframe, target_size, dataset_type="train"):
12         self.dataframe = dataframe
13         self.target_size = target_size
14         self.dataset_type = dataset_type
15
16         self.to_tensor = transforms.ToTensor()
17
18         # Define the normalization transform
19         self.normalize = transforms.Normalize(mean=[0.485, 0.456, 0.406],
20                                               std=[0.229, 0.224, 0.225])
21
22     def __len__(self):
23         return len(self.dataframe)
24
25     def __getitem__(self, idx):
26         row = self.dataframe.iloc[idx]
27         image = Image.open(row['image_path']).convert('RGB')
28         annotation = Image.open(row['annotation_path']).convert('L')
29         annotation_rgb = Image.open(row['annotation_rgb_path']).convert('RGB')
30         annotation_mask = np.load(row['annotation_mask_path'])
31         id = row['id']
32         material = row['material']
33         content = row['content']
34
35         if self.dataset_type == "train":
36             # Generate random square cropping coordinates
37             crop_coords = random_square_crop_params(image, self.target_size)
38
39             # Apply the same cropping and resizing to all
40             image = apply_crop_and_resize(image, crop_coords, self.target_size)
41             annotation = apply_crop_and_resize(annotation, crop_coords, self.target_size)
42             annotation_rgb = apply_crop_and_resize(annotation_rgb, crop_coords, self.target_size)
43             annotation_mask = np.array(apply_crop_and_resize(transforms.functional.to_pil_image(annotation_mask), crop_coords, self.target_size))
44
45         elif self.dataset_type == "validation": # Validation
46             max_edge = max(image.size)
47             if max_edge > 1024:
48                 downsample_ratio = 1024 / max_edge
49                 new_size = tuple([int(dim * downsample_ratio) for dim in image.size])
50
51                 image = image.resize(new_size, Image.BILINEAR)
52                 annotation = annotation.resize(new_size, Image.NEAREST)
53                 annotation_rgb = annotation_rgb.resize(new_size, Image.BILINEAR)
54                 annotation_mask = np.array(transforms.functional.to_pil_image(annotation_mask).resize(new_size, Image.NEAREST))
55
56         elif self.dataset_type == "test": # Test
57             # Instead of cropping, downsize the images so that the longest edge is 1024 or less
58             max_edge = max(image.size)
59             if max_edge > 1024:
60                 downsample_ratio = 1024 / max_edge
61                 new_size = tuple([int(dim * downsample_ratio) for dim in image.size])
62
63                 image = image.resize(new_size, Image.BILINEAR)
64                 annotation = annotation.resize(new_size, Image.NEAREST)
65                 annotation_rgb = annotation_rgb.resize(new_size, Image.BILINEAR)
66                 annotation_mask = np.array(transforms.functional.to_pil_image(annotation_mask).resize(new_size, Image.NEAREST))
67
68         # Convert PIL images to PyTorch tensors
69         image = self.to_tensor(image)
70         annotation = torch.tensor(np.array(annotation), dtype=torch.long)
71         annotation_rgb = self.to_tensor(annotation_rgb)
72         annotation_mask = torch.from_numpy(annotation_mask).long()
73
74         # Normalize the image
75         image = self.normalize(image)
76
77         # Change all values in annotation that are 255 to 16

```

```

78         annotation[annotation == 255] = 16
79
80     return {
81         'image': image,
82         'annotation': annotation,
83         'annotation_rgb': annotation_rgb,
84         'annotation_mask': annotation_mask,
85         'id': id,
86         'material': material,
87         'content': content
88     }

```

➤ 2. CustomDataset with Data-augmentation

[] ↳ 1 cell hidden

▼ CustomDataModule

Here we create a DataModule which encapsulates our training and validation DataLoaders; you can also do this manually by only using the Pytorch DataLoader class, lines 24 and 27.

```

1 from torch.utils.data import DataLoader
2
3 class CustomDataModule(pl.LightningDataModule):
4     def __init__(self, loocv_splits, current_material, target_size, batch_size=32, num_workers=4):
5         super().__init__()
6         self.loocv_splits = loocv_splits
7         self.current_material = current_material
8         self.target_size = target_size
9         self.batch_size = batch_size
10        self.num_workers = num_workers
11
12    def prepare_data(self):
13        pass
14
15    def setup(self, stage=None):
16        # Load current train and validation set based on LOOCV iteration
17        train_df = self.loocv_splits[self.current_material]['train_set']
18        val_df = self.loocv_splits[self.current_material]['val_set'].sample(frac=1).reset_index(drop=True)
19
20        self.train_dataset = CustomDataset(dataframe=train_df, target_size=self.target_size, dataset_type="train")
21        self.val_dataset = CustomDataset(dataframe=val_df, target_size=self.target_size, dataset_type="validation")
22        self.test_dataset = CustomDataset(dataframe=val_df, target_size=self.target_size, dataset_type="test")
23
24    def train_dataloader(self):
25        return DataLoader(self.train_dataset, batch_size=self.batch_size, shuffle=True, num_workers=self.num_workers)
26
27    def val_dataloader(self):
28        return DataLoader(self.val_dataset, batch_size=1, shuffle=False, num_workers=self.num_workers)
29
30    def test_dataloader(self):
31        return DataLoader(self.test_dataset, batch_size=1, shuffle=False, num_workers=self.num_workers)
32

```

➤ Dataset visualisation

[] ↳ 10 cells hidden

▼ Train Model

We are planning to do semantic segmentation for the coursework. Hence, we will be using Fully Convolutional Networks to retain the spatial information.

▼ Model architectures

➤ 1. VGGUNet

When the model is trained for the task of semantic segmentation, the encoder outputs a tensor containing information about the objects, and its shape and size. The decoder takes this information and produces the segmentation maps.

In this coursework, we will be utilizing pretrained vgg16 models with UNET architecture.

[] ↳ 3 cells hidden

➤ 2. MobileNetV2

[] ↳ 3 cells hidden

➤ 3. DeepLabV3

[] ↳ 4 cells hidden

▼ Model Training

Adjust parameters (ex.batch size and learning rate)

```
1 # 1. VGGUNet
2 # target_size = 512
3 # batch_size = 16
4 # learning_rate = 0.0005
5 # num_epochs = 30
6
7 # 2. MobileNetV2
8 # target_size = 512
9 # batch_size = 64
10 # learning_rate = 0.0001
11 # num_epochs = 50
12
13 # 3. DeepLabV3 ResNet50
14 # target_size = 512
15 # batch_size = 21
16 # learning_rate = 0.00009
17 # num_epochs = 100
18
19 # 4. DeepLabV3 ResNet101
20 target_size = 512
21 batch_size = 13
22 learning_rate = 0.0001
23 num_epochs = 100
```

Create a new data module

```
1 data_module = CustomDataModule(loocv_splits=loocv_splits,
2                               current_material=unique_material[2],
3                               target_size=target_size,
4                               batch_size=batch_size)
5 data_module.setup()
6 train_loader = data_module.train_dataloader()
7 val_loader = data_module.val_dataloader()
```

▼ Initialize Loss

1. Cross-Entropy Loss

```
1 criterion = nn.CrossEntropyLoss()
```

2. Dice Loss

```

1 class MultiClassDiceLoss(nn.Module):
2     def __init__(self, smooth=1., ignore_index=None):
3         super(MultiClassDiceLoss, self).__init__()
4         self.smooth = smooth
5         self.ignore_index = ignore_index
6
7     def forward(self, inputs, targets):
8         # inputs: [N, C, H, W] Variable with logits (not probabilities) from the model
9         # targets: [N, H, W] LongTensor with class indices
10        inputs = torch.softmax(inputs, dim=1)
11
12        C = inputs.shape[1]
13        dice_loss = 0.
14        for c in range(C):
15            if c != self.ignore_index: # Optional: Ignore a certain class index
16                target_c = (targets == c).float()
17                input_c = inputs[:, c]
18                intersection = (input_c * target_c).sum()
19                dice_c = (2. * intersection + self.smooth) / (input_c.sum() + target_c.sum() + self.smooth)
20                dice_loss += (1 - dice_c)
21
22        return dice_loss / (C - (1 if self.ignore_index is not None else 0))
23
24 if run_this_cell:
25     # Initialize the combined loss
26     criterion = MultiClassDiceLoss(smooth=1.0)

```

3. Combine Dice and Cross-Entropy Loss

```

1 class CombinedDiceCrossEntropyLoss(nn.Module):
2     def __init__(self, dice_smooth=1., dice_weight=0.5, ce_weight=0.5, ignore_index=None):
3         super(CombinedDiceCrossEntropyLoss, self).__init__()
4         self.dice_smooth = dice_smooth
5         self.dice_weight = dice_weight
6         self.ce_weight = ce_weight
7         self.ignore_index = ignore_index
8
9     def forward(self, inputs, targets):
10        # Compute Dice Loss
11        inputs_softmax = torch.softmax(inputs, dim=1)
12        C = inputs_softmax.shape[1]
13        dice_loss = 0.
14        for c in range(C):
15            if c != self.ignore_index:
16                target_c = (targets == c).float()
17                input_c = inputs_softmax[:, c]
18                intersection = (input_c * target_c).sum()
19                dice_c = (2. * intersection + self.dice_smooth) / (input_c.sum() + target_c.sum() + self.dice_smooth)
20                dice_loss += (1 - dice_c)
21        dice_loss /= (C - (1 if self.ignore_index is not None else 0))
22
23        # Compute CrossEntropy Loss with conditional ignore_index
24        if self.ignore_index is not None:
25            ce_loss = F.cross_entropy(inputs, targets, ignore_index=self.ignore_index)
26        else:
27            ce_loss = F.cross_entropy(inputs, targets)
28
29        # Combine the losses
30        combined_loss = self.dice_weight * dice_loss + self.ce_weight * ce_loss
31        return combined_loss
32
33 if run_this_cell:
34     # Example usage
35     criterion = CombinedDiceCrossEntropyLoss(dice_smooth=1.0, dice_weight=0.4, ce_weight=0.6, ignore_index=None)

```

> Train the Model

Since we are doing segmetation task and segmentation is a classification problem at each pixel, the suitable loss function is Cross-Entropy Loss `nn.CrossEntropyLoss`, which is commonly used for multi-class classification tasks.

```
[ ] ↳ 1 cell hidden
```

✓ Train the Model with Mix Precision Training

```
1 if run_this_cell:
2     # Store training and validation loss
3     train_losses = []
4     validation_losses = []
5
6     # VGGUNet
7     # optimizer = torch.optim.Adam(filter(lambda p: p.requires_grad, model.parameters()), lr=learning_rate)
8
9     # Others
10    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
11
12    # Initialize the learning rate scheduler
13    # scheduler = StepLR(optimizer, step_size=10, gamma=0.1)
14    scheduler = ReduceLROnPlateau(optimizer, 'min', patience=3, factor=0.7, verbose=True)
15
16    # Early stopping initialization
17    best_val_loss = float('inf')
18    patience = 10
19    patience_counter = 0
20
21    # Initialize the gradient scaler
22    scaler = GradScaler()
23
24    # Example training loop
25    for epoch in range(num_epochs):
26        model.train()
27        train_loss = 0.0
28        for batch in train_loader:
29            inputs = batch['image'].to(DEVICE)
30            annotation_masks = batch['annotation_mask'].to(DEVICE)
31
32            optimizer.zero_grad()
33
34            # Forward pass with autocast
35            with autocast():
36                # outputs = model(inputs)          # Other models
37                outputs = model(inputs)['out']     # DeepLabV3
38                loss = criterion(outputs, annotation_masks)
39
40            # Backward and optimize
41            scaler.scale(loss).backward()
42            scaler.step(optimizer)
43            scaler.update()
44
45            train_loss += loss.item() * inputs.size(0)
46
47        # Calculate average losses
48        train_loss = train_loss / len(train_loader.dataset)
49
50        # Validation phase
51        model.eval() # Set the model to evaluation mode
52        val_loss = 0.0
53        with torch.no_grad():
54            for batch in val_loader:
55                inputs = batch['image'].to(DEVICE)
56                annotation_masks = batch['annotation_mask'].to(DEVICE)
57
58                # Forward pass with autocast
59                with autocast():
60                    # outputs = model(inputs)          # Other models
61                    outputs = model(inputs)['out']     # DeepLabV3
62                    loss = criterion(outputs, annotation_masks)
63                    preds = torch.softmax(outputs, dim=1) # Convert logits to probabilities
64
65                val_loss += loss.item() * inputs.size(0)
66
67        # Calculate average losses
68        val_loss = val_loss / len(val_loader.dataset)
69
70        # Append train loss and validation loss
71        train_losses.append(np.round(train_loss, 6))
72        validation_losses.append(np.round(val_loss, 6))
73
```

```

74     best_model_so_far = True
75     # Early stopping check
76     if val_loss < best_val_loss:
77         best_val_loss = val_loss
78         patience_counter = 0 # Reset patience counter
79         temp_model = model
80         best_model_so_far = True
81     else:
82         patience_counter += 1 # Increment patience counter
83         best_model_so_far = False
84
85     # Save the best acquired model training so far
86     if patience_counter == 1:
87         model_save = f'deepabv3_resnet100_model-epoch{epoch}.pth'
88         torch.save(temp_model, model_save)
89
90     # Print training/validation statistics
91     if best_model_so_far:
92         print(f'Epoch: {epoch+1}/{num_epochs} \tTraining Loss: {train_loss:.6f} \tValidation Loss: {val_loss:.6f} (Best so far!)')
93     else:
94         print(f'Epoch: {epoch+1}/{num_epochs} \tTraining Loss: {train_loss:.6f} \tValidation Loss: {val_loss:.6f}')
95
96     # Step the scheduler
97     # scheduler.step()
98     scheduler.step(val_loss)
99
100     if patience_counter >= patience:
101         print(f"Early stopping triggered after {epoch+1} epochs.")
102         break
103

```

 /usr/local/lib/python3.10/dist-packages/torch/optim/lr_scheduler.py:28: UserWarning: The verbose parameter is deprecated. Please use get_warnings.warn("The verbose parameter is deprecated. Please use get_last_lr() "

Epoch: 1/100	Training Loss: 2.543830	Validation Loss: 2.281083 (Best so far!)
Epoch: 2/100	Training Loss: 2.125703	Validation Loss: 3.204621
Epoch: 3/100	Training Loss: 1.768130	Validation Loss: 1.952916 (Best so far!)
Epoch: 4/100	Training Loss: 1.513417	Validation Loss: 1.507389 (Best so far!)
Epoch: 5/100	Training Loss: 1.278513	Validation Loss: 1.375146 (Best so far!)
Epoch: 6/100	Training Loss: 1.119651	Validation Loss: 1.214672 (Best so far!)
Epoch: 7/100	Training Loss: 0.987867	Validation Loss: 1.108479 (Best so far!)
Epoch: 8/100	Training Loss: 0.862274	Validation Loss: 0.958986 (Best so far!)
Epoch: 9/100	Training Loss: 0.764531	Validation Loss: 0.866033 (Best so far!)
Epoch: 10/100	Training Loss: 0.700443	Validation Loss: 0.884067
Epoch: 11/100	Training Loss: 0.623949	Validation Loss: 0.772865 (Best so far!)
Epoch: 12/100	Training Loss: 0.565045	Validation Loss: 0.738967 (Best so far!)
Epoch: 13/100	Training Loss: 0.518091	Validation Loss: 0.781551
Epoch: 14/100	Training Loss: 0.500389	Validation Loss: 0.720902 (Best so far!)
Epoch: 15/100	Training Loss: 0.465065	Validation Loss: 0.695185 (Best so far!)
Epoch: 16/100	Training Loss: 0.433149	Validation Loss: 0.691596 (Best so far!)
Epoch: 17/100	Training Loss: 0.422272	Validation Loss: 0.657719 (Best so far!)
Epoch: 18/100	Training Loss: 0.391193	Validation Loss: 0.663207
Epoch: 19/100	Training Loss: 0.388699	Validation Loss: 0.692254
Epoch: 20/100	Training Loss: 0.379328	Validation Loss: 0.655263 (Best so far!)
Epoch: 21/100	Training Loss: 0.386421	Validation Loss: 0.675769
Epoch: 22/100	Training Loss: 0.335168	Validation Loss: 0.677437
Epoch: 23/100	Training Loss: 0.363550	Validation Loss: 0.642111 (Best so far!)
Epoch: 24/100	Training Loss: 0.337744	Validation Loss: 0.624480 (Best so far!)
Epoch: 25/100	Training Loss: 0.333786	Validation Loss: 0.624097 (Best so far!)
Epoch: 26/100	Training Loss: 0.322835	Validation Loss: 0.650784
Epoch: 27/100	Training Loss: 0.354497	Validation Loss: 0.641935
Epoch: 28/100	Training Loss: 0.309550	Validation Loss: 0.621830 (Best so far!)
Epoch: 29/100	Training Loss: 0.294612	Validation Loss: 0.624972
Epoch: 30/100	Training Loss: 0.303154	Validation Loss: 0.659039
Epoch: 31/100	Training Loss: 0.293801	Validation Loss: 0.652622
Epoch: 32/100	Training Loss: 0.305460	Validation Loss: 0.626309
Epoch: 33/100	Training Loss: 0.283875	Validation Loss: 0.639668
Epoch: 34/100	Training Loss: 0.276685	Validation Loss: 0.613244 (Best so far!)
Epoch: 35/100	Training Loss: 0.271926	Validation Loss: 0.598881 (Best so far!)
Epoch: 36/100	Training Loss: 0.262117	Validation Loss: 0.617369
Epoch: 37/100	Training Loss: 0.240675	Validation Loss: 0.608249
Epoch: 38/100	Training Loss: 0.256647	Validation Loss: 0.667266
Epoch: 39/100	Training Loss: 0.240438	Validation Loss: 0.646031
Epoch: 40/100	Training Loss: 0.251889	Validation Loss: 0.608699
Epoch: 41/100	Training Loss: 0.238749	Validation Loss: 0.606095
Epoch: 42/100	Training Loss: 0.242850	Validation Loss: 0.600584


```

4         batch_size=batch_size)
5
6 test_module.setup()
7
8 test_loader = test_module.test_dataloader()
9

```

➤ Visualise Test Samples

I have modified the visualisation code given to visualise the images in the test samples

```
[ ] 1 cell hidden
```

✓ Predict Test Dataset

Load pre-trained autoencoder model

```

1 # Other models
2 # model = torch.load("mobilenet_model.pth")
3
4 # DeepLabV3
5 model = torch.load("./damage_data/deeplabv3_resnet100_model2-epoch22.pth", map_location=torch.device('cpu')) # CPU
6 # model = torch.load("deeplabv3_resnet50_model-epoch59.pth") # GPU
7
8 model.eval()

DeepLabV3(
  (backbone): IntermediateLayerGetter(
    (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
    (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
    (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
    (layer1): Sequential(
      (0): Bottleneck(
        (conv1): Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (relu): ReLU(inplace=True)
        (downsample): Sequential(
          (0): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
          (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        )
      )
    )
    (1): Bottleneck(
      (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (2): Bottleneck(
      (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
  )
  (layer2): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(256, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
  )
)

```

```

    )
)
(1): Bottleneck(
  (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
  (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)

```

Predict with test dataset and compute the loss

```

1 # criterion = nn.CrossEntropyLoss()
2
3 test_loss = 0.0
4 with torch.no_grad():
5     for batch in test_loader:
6         # inputs = batch['image'].to(DEVICE)
7         # annotation_masks = batch['annotation_mask'].to(DEVICE)
8         inputs = batch['image']
9         annotation_masks = batch['annotation_mask']
10
11     # Forward pass
12     # outputs = model(inputs)          # Other models
13     outputs = model(inputs)['out']    # DeepLabV3
14     loss = criterion(outputs, annotation_masks)
15
16     test_loss += loss.item() * inputs.size(0)
17
18 # Calculate average losses
19 test_loss = test_loss / len(test_loader.dataset)
20
21 # Print test statistics
22 print(f'Testing Loss: {test_loss:.6f}')

```

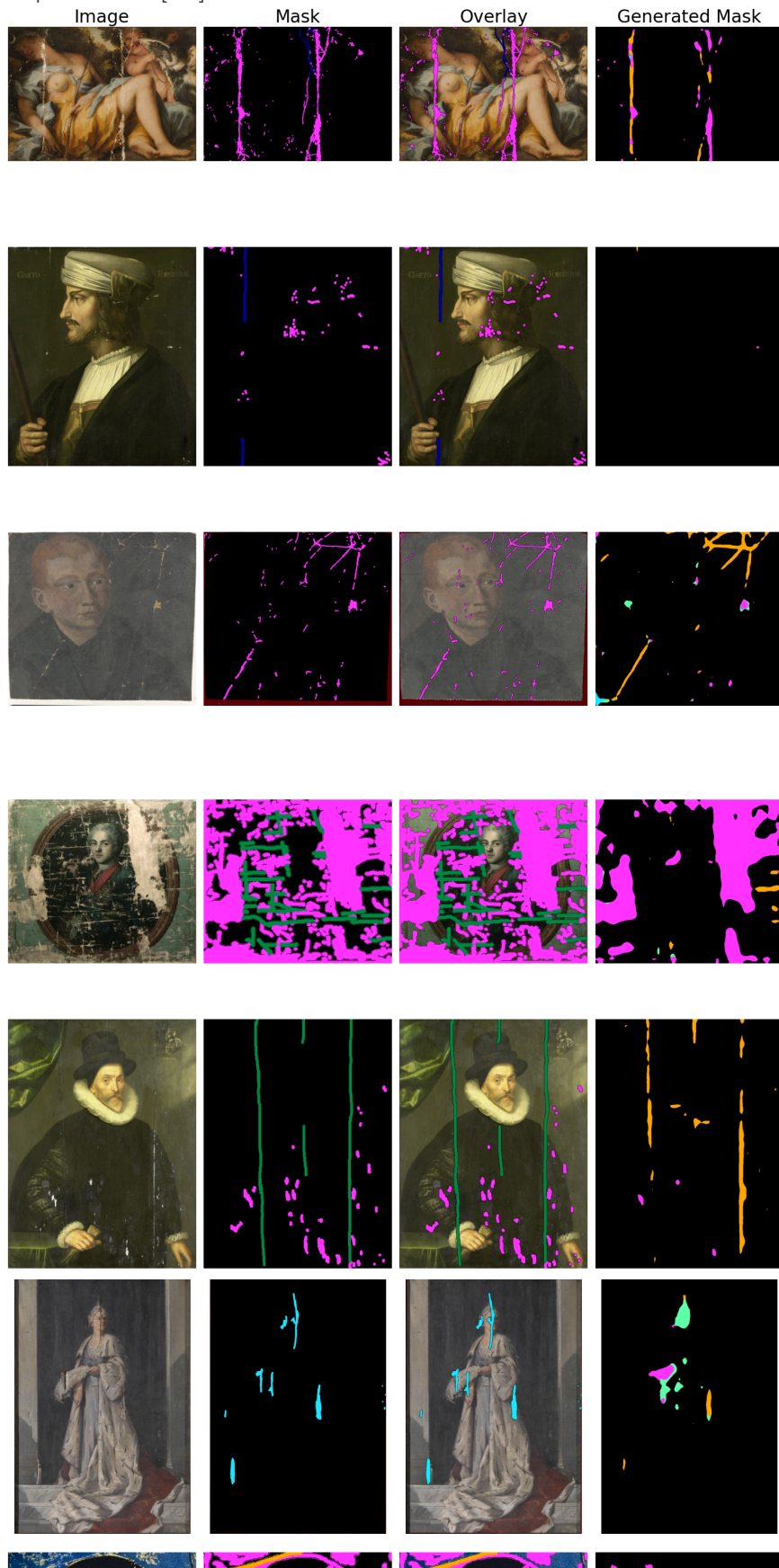
✓ Visualise Generated Output Images

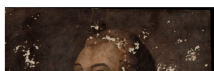
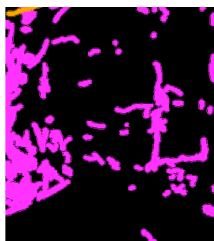
```

1 # Number of examples to visualize
2 N = 9
3
4 test_iter = iter(test_loader)
5 # for _ in range(8):
6 #     next(test_iter)
7 example_batches = [next(test_iter) for _ in range(N)]
8
9 # Initialize empty lists to collect different parts of each batch
10 example_images = []
11 example_annotations = []
12 example_annotation_rgbs = []
13 example_annotation_masks = []
14 example_generated_annotation_masks = []
15 example_materials = []
16 example_contents = []
17
18 # Populate the lists with the data from the 4 batches
19 for batch in example_batches:
20     example_images.append(batch['image'].squeeze())
21     example_annotations.append(batch['annotation'].squeeze())
22     example_annotation_rgbs.append(batch['annotation_rgb'].squeeze())
23     example_annotation_masks.append(batch['annotation_mask'].squeeze())
24     # example_generated_annotation_masks.append(model(batch['image'].to(DEVICE))) # Other models
25     # example_generated_annotation_masks.append(model(batch['image'].to(DEVICE))['out']) # DeepLabV3
26     example_generated_annotation_masks.append(model(batch['image'])['out']) # DeepLabV3
27     example_materials.append(batch['material'][0])
28     example_contents.append(batch['content'][0])
29
30 fig, axes = plt.subplots(N, 4, figsize=(15, 5 * N))
31
32 for ax, col in zip(axes[0], ['Image', 'Mask', 'Overlay', 'Generated Mask']):
33     ax.set_title(col, fontsize=24)
34
35 for i in range(N):
36     example_image = denormalize(example_images[i].numpy().transpose((1, 2, 0)), mean, std) # C, H, W -> H, W, C
37     example_annotation = example_annotations[i].numpy()
38     example_annotation_rgb = example_annotation_rgbs[i].numpy().transpose((1, 2, 0)) # C, H, W -> H, W, C
39
40     # Convert annotation mask to rgb image for visualisation
41     example_annotation_mask = mask_to_rgb_image(example_annotation_masks[i])
42
43     # Argmax prediction_one_hot of the generated mask
44     # example_generated_annotation_mask = torch.argmax(example_generated_annotation_masks[i], dim=1).squeeze().numpy() # Using pytor
45     example_generated_annotation_mask = np.argmax(example_generated_annotation_masks[i].detach().numpy(), axis=1).squeeze() # CPU
46     # example_generated_annotation_mask = np.argmax(example_generated_annotation_masks[i].cpu().detach().numpy(), axis=1).squeeze() # GPU
47
48     print(f"Unique Class ID: {np.unique(example_generated_annotation_mask)}")
49     example_generated_annotation_mask = mask_to_rgb_image(example_generated_annotation_mask)
50
51     example_material = example_materials[i]
52     example_content = example_contents[i]
53     # Create an alpha (transparency) channel where black pixels in annotation_rgb are fully transparent
54     alpha_channel = np.all(example_annotation_rgb == [0, 0, 0], axis=-1)
55     example_annotation_rgba = np.dstack((example_annotation_rgb, np.where(alpha_channel, 0, 1)))
56     axes[i, 0].imshow(example_image)
57     axes[i, 0].axis('off')
58
59     axes[i, 1].imshow(example_annotation_mask)
60     axes[i, 1].axis('off')
61
62     axes[i, 2].imshow(example_image)
63     axes[i, 2].imshow(example_annotation_rgba)
64     axes[i, 2].axis('off')
65
66     axes[i, 3].imshow(example_generated_annotation_mask)
67     axes[i, 3].axis('off')
68
69 plt.tight_layout()
70 plt.show()

```

Unique Class ID: [0 1 2 7]
Unique Class ID: [0 2 7]
Unique Class ID: [0 1 2 7 10 16]
Unique Class ID: [0 2 7 10]
Unique Class ID: [0 2 7]
Unique Class ID: [0 2 7 10]
Unique Class ID: [0 2]
Unique Class ID: [0 2]
Unique Class ID: [0 7]






```

1 # Number of examples to visualize
2 N = 23
3
4 test_iter = iter(test_loader)
5 example_batches = [next(test_iter) for _ in range(N)]
6
7 # Initialize empty lists to collect different parts of each batch
8 example_images = []
9 example_annotations = []
10 example_annotation_rgbs = []
11 example_annotation_masks = []
12 example_generated_annotation_masks = []
13 example_materials = []
14 example_contents = []
15
16 # Populate the lists with the data from the 4 batches
17 for batch in example_batches:
18     example_images.append(batch['image'].squeeze())
19     example_annotations.append(batch['annotation'].squeeze())
20     example_annotation_rgbs.append(batch['annotation_rgb'].squeeze())
21     example_annotation_masks.append(batch['annotation_mask'].squeeze())
22     # example_generated_annotation_masks.append(model(batch['image'].to(DEVICE))) # Other models
23     # example_generated_annotation_masks.append(model(batch['image'].to(DEVICE))['out']) # DeepLabV3
24     example_generated_annotation_masks.append(model(batch['image'])['out']) # DeepLabV3
25     example_materials.append(batch['material'][0])
26     example_contents.append(batch['content'][0])
27
28 fig, axes = plt.subplots(N, 4, figsize=(15, 5 * N))
29
30 for ax, col in zip(axes[0], ['Image', 'Mask', 'Overlay', 'Generated Mask']):
31     ax.set_title(col, fontsize=24)
32
33 for i in range(N):
34     example_image = denormalize(example_images[i].numpy().transpose((1, 2, 0)), mean, std) # C, H, W -> H, W, C
35     example_annotation = example_annotations[i].numpy()
36     example_annotation_rgb = example_annotation_rgbs[i].numpy().transpose((1, 2, 0)) # C, H, W -> H, W, C
37
38     # Convert annotation mask to rgb image for visualisation
39     example_annotation_mask = mask_to_rgb_image(example_annotation_masks[i])
40
41     # Argmax prediction_one_hot of the generated mask
42     # example_generated_annotation_mask = torch.argmax(example_generated_annotation_masks[i], dim=1).squeeze().numpy() # Using pytor
43     example_generated_annotation_mask = np.argmax(example_generated_annotation_masks[i].detach().numpy(), axis=1).squeeze() # CPU
44     # example_generated_annotation_mask = np.argmax(example_generated_annotation_masks[i].cpu().detach().numpy(), axis=1).squeeze() # GPU
45
46     print(f"Unique Class ID: {np.unique(example_generated_annotation_mask)}")
47     example_generated_annotation_mask = mask_to_rgb_image(example_generated_annotation_mask)
48
49     example_material = example_materials[i]
50     example_content = example_contents[i]
51     # Create an alpha (transparency) channel where black pixels in annotation_rgb are fully transparent
52     alpha_channel = np.all(example_annotation_rgb == [0, 0, 0], axis=-1)
53     example_annotation_rgba = np.dstack((example_annotation_rgb, np.where(alpha_channel, 0, 1)))
54     axes[i, 0].imshow(example_image)
55     axes[i, 0].axis('off')
56
57     axes[i, 1].imshow(example_annotation_mask)
58     axes[i, 1].axis('off')
59
60     axes[i, 2].imshow(example_image)
61     axes[i, 2].imshow(example_annotation_rgba)
62     axes[i, 2].axis('off')
63
64     axes[i, 3].imshow(example_generated_annotation_mask)
65     axes[i, 3].axis('off')
66
67 plt.tight_layout()
68 plt.show()

```